

AWS SAA-C03

Databases

RDS, Aurora, DynamoDB, ElastiCache, Redshift

Solutions Architect Associate

Dispensa di Studio Approfondita

Amazon RDS: database relazionali gestiti

Amazon RDS (Relational Database Service) elimina il lavoro operativo di gestire un database relazionale: provisioning hardware, patching, backup, replica. Tu ti concentri sullo schema e sulle query, AWS si occupa dell'infrastruttura.

RDS supporta sei engine: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server e IBM Db2. La scelta dell'engine dipende dalle tue esigenze applicative e dalle licenze esistenti.

Multi-AZ: alta disponibilit 

Multi-AZ crea una replica sincrona del database in un'altra Availability Zone. La replica standby NON   accessibile per le letture: esiste solo per il failover. Se l'istanza primaria ha un problema, RDS esegue automaticamente il failover verso lo standby in 1-2 minuti. Il DNS endpoint rimane lo stesso, quindi l'applicazione non deve essere modificata.

Multi-AZ   per alta disponibilit , non per performance. Se hai bisogno di scalare le letture, usa le Read Replicas.

Read Replicas: scalare le letture

Le Read Replicas sono copie asincrone del database che puoi usare per le query di lettura. Puoi creare fino a 15 repliche per un database RDS. Ogni replica ha il suo endpoint DNS. L'applicazione deve essere modificata per inviare le letture alle repliche e le scritture al primario.

Le repliche possono essere nella stessa regione, in una regione diversa (Cross-Region Read Replica) o addirittura promosse a database standalone indipendente. La replica cross-region   utile per disaster recovery e per ridurre la latenza per utenti in altre regioni.

SUGGERIMENTO ESAME: "Migliorare le performance di lettura" = Read Replicas. "Alta disponibilit  con failover automatico" = Multi-AZ. Spesso la risposta corretta   entrambi.

Backup, crittografia e RDS Proxy

I backup automatici creano uno snapshot giornaliero e salvano i transaction log ogni 5 minuti. Questo permette il point-in-time recovery: puoi ripristinare il database a qualsiasi secondo negli ultimi 1-35 giorni (configurabile). Gli snapshot manuali persistono anche dopo la cancellazione del database.

La crittografia at-rest usa KMS e deve essere abilitata alla creazione del database. Non puoi criptare un database esistente direttamente: devi creare uno snapshot, copiarlo con crittografia abilitata, e ripristinare da quello.

RDS Proxy   un servizio di connection pooling che si posiziona tra l'applicazione e il database. Mantiene un pool di

connessioni aperte verso RDS, riducendo il tempo di failover del 66% e gestendo meglio i picchi di connessioni. E' particolarmente utile con Lambda, dove ogni invocazione potrebbe aprire una nuova connessione.

Amazon Aurora: il database cloud-native

Aurora è il database relazionale progettato da AWS specificamente per il cloud. È compatibile con MySQL e PostgreSQL (puoi usare gli stessi driver e strumenti), ma l'architettura sottostante è completamente diversa e offre performance fino a 5 volte superiori a MySQL e 3 volte superiori a PostgreSQL.

L'architettura di Aurora separa compute e storage. Lo storage è distribuito su 3 AZ con 6 copie dei dati. Aurora può continuare a funzionare anche perdendo 2 copie per le scritture o 3 copie per le letture. Lo storage scala automaticamente da 10GB a 128TB senza intervento.

Aurora ha due tipi di endpoint: il Writer Endpoint punta sempre all'istanza primaria (per le scritture), il Reader Endpoint fa load balancing tra tutte le Read Replicas (fino a 15). Questo semplifica la configurazione dell'applicazione.

Aurora Serverless e Global Database

Aurora Serverless v2 scala automaticamente la capacità di calcolo in base al carico. La capacità è misurata in ACU (Aurora Capacity Units) e scala in incrementi di 0.5 ACU. Paghi per gli ACU effettivamente usati al secondo. È ideale per workload imprevedibili, ambienti di sviluppo, e applicazioni con traffico intermittente.

Aurora Global Database replica i dati in fino a 5 regioni secondarie con un lag tipico inferiore a 1 secondo. In caso di disastro regionale, puoi promuovere una regione secondaria a primaria in meno di 1 minuto. È la soluzione per disaster recovery cross-region con RPO e RTO minimi.

SUGGERIMENTO ESAME: "Database relazionale con massime performance e disponibilità" = Aurora. "Workload imprevedibile senza gestire capacità" = Aurora Serverless. "Disaster recovery cross-region con RPO < 1 secondo" = Aurora Global Database.

Amazon DynamoDB: NoSQL serverless

DynamoDB e' un database NoSQL key-value e document completamente serverless. Non ci sono server da gestire, la capacita' scala automaticamente, e la latenza e' single-digit millisecond a qualsiasi scala.

Ogni tabella ha una partition key (obbligatoria) che determina come i dati sono distribuiti. Puoi aggiungere una sort key per creare chiavi composite e permettere query range. La scelta delle chiavi e' critica per le performance: una partition key con alta cardinalita' distribuisce il carico uniformemente.

Capacity modes e DAX

DynamoDB offre due modalita' di capacita'. On-Demand paga per ogni lettura e scrittura, senza provisioning. E' ideale per workload imprevedibili o nuove applicazioni. Provisioned definisce RCU (Read Capacity Units) e WCU (Write Capacity Units) in anticipo, con la possibilita' di abilitare auto-scaling. E' piu' economico per workload prevedibili.

DynamoDB Accelerator (DAX) e' una cache in-memory completamente compatibile con l'API DynamoDB. Riduce la latenza da millisecondi a microsecondi per le letture. L'applicazione punta a DAX invece che a DynamoDB, e DAX gestisce la cache in modo trasparente.

Global Tables e Streams

Global Tables replica automaticamente una tabella DynamoDB in piu' regioni con un modello active-active: puoi scrivere in qualsiasi regione e le modifiche si propagano alle altre. Richiede DynamoDB Streams abilitato.

DynamoDB Streams cattura ogni modifica alla tabella (insert, update, delete) in un log ordinato. Puoi collegare una funzione Lambda che reagisce alle modifiche in tempo reale. Casi d'uso: trigger per elaborazioni, replica verso altri sistemi, audit log.

SUGGERIMENTO ESAME: "Database NoSQL serverless con latenza garantita" = DynamoDB. "Cache per DynamoDB" = DAX. "Replica multi-region active-active" = Global Tables.

ElastiCache: cache in-memory

ElastiCache offre due engine: Redis e Memcached. La scelta dipende dalle tue esigenze.

Redis supporta strutture dati avanzate (liste, set, sorted set, hash), persistence su disco, replica con failover automatico, cluster mode per sharding, pub/sub messaging e scripting Lua. E' la scelta per la maggior parte dei casi d'uso: session store, leaderboard, geospatial, caching complesso.

Memcached e' piu' semplice: puro key-value caching, multi-threaded, nessuna persistence o replica. E' la scelta quando hai bisogno solo di caching semplice e vuoi sfruttare il multi-threading.

Le strategie di caching piu' comuni sono Lazy Loading (carica in cache solo quando richiesto e non presente) e Write-Through (scrivi in cache ad ogni scrittura nel database). Lazy Loading puo' servire dati stale, Write-Through ha latenza di scrittura piu' alta.

SUGGERIMENTO ESAME: "Cache con alta disponibilita' e persistence" = Redis. "Caching semplice multi-threaded" = Memcached.

Amazon Redshift: data warehouse

Redshift e' un data warehouse colonnare basato su PostgreSQL, progettato per analytics su petabyte di dati. A differenza di RDS che e' ottimizzato per transazioni (OLTP), Redshift e' ottimizzato per query analitiche complesse su grandi volumi di dati (OLAP).

Redshift Spectrum permette di eseguire query su dati in S3 senza caricarli in Redshift. Puoi unire dati in Redshift con dati in S3 nella stessa query. Redshift Serverless scala automaticamente la capacita' senza gestire cluster.

Redshift NON e' Multi-AZ. Per alta disponibilita', usa cluster multi-nodo (i dati sono replicati tra i nodi) e snapshot automatici, che possono essere copiati in altre regioni.

SUGGERIMENTO ESAME: "Analytics su petabyte di dati" = Redshift. "Query su dati in S3 senza caricarli" = Redshift Spectrum.